Final Proposed Schema & Implementation Procedure — Historical APR Void + Full-Year Replay — RECONCILED v6

Status: PROPOSAL ONLY — **FOR DISCUSSION ONLY — DO NOT EXECUTE DDL OR CODE CHANGES UNTIL MUTUAL APPROVAL**
Date: 2026-09-01 — **Author:** Jose — **For:** Hal & Rick — **RECONCILED v6 (BravoFrontend ffdea5c + 11-point review §§1-11 reconciled, 7 open items closed)**
Version: `v6` — Decisions 3,5,6 + reconciled §§1-3,5-7 (full-year supersede all, CreditLedger EventDate/Status, composite CF identity, tenant isolation, deterministic backfill, FY baseline)
Decisions incorporated: F1-F4 in scope, Full-year replay, Source + SubmissionKey, CreditLedger separate table, Supersede, Lean Batch, CashFlow REVISED, **Split-Bank Routing & AssessmentBankAssignmentHistory reconciled — Business Rule: Separate physical CashFlow per BankID, No Views (accounting principle)**
Refs: `backend/migrations/002_apr_historical_void.sql` (proposal), `backend/server.js:3955-4621`, Figures 1-3

---

1. Executive Summary — RECONCILED with BravoFrontend split-bank (Rick §§4-5, 2026-09-01)

After Rick's approval (8 points) and the Architecture Update (BravoFrontend ffdea5c), the final procedure **reconciled** is:

- **Full fiscal-year rebuild** from immutable `AssessmentPaymentSource` (not intermediate balance reconstruction).
- **Source is immutable** (only `Status='VOID'` changes); determines overflow rules `SA→AD→Credit` vs `AD→Credit`.
- **SubmissionKey groups the originating UF submission, but is NOT one physical CashFlow receipt.** Verified 001006 `APR090126-11074218` $600 → **two receipts**: `$500→SA bank Capital(2)` + `$100→AD bank Operating(1)` sharing one `SubmissionKey` + `TransactionNumber`. Physical receipt is therefore per `(SubmissionKey, BankAccountID)` (Rick §5).
- **Split-bank routing (BravoFrontend §§3-4):** SA dollars → SA programmed bank, SA overflow → AD → AD bank, Credit → AD bank, AD dollars → AD bank. DuesProgramming + `AssessmentBankAssignmentHistory` is authority; replay must resolve bank effective on `PaymentDate`, not today's bank.
- **Supersede, not edit:** old APR rows become `SUPERSEDED`, new rows reuse same `TransactionNumber`.
- **Separate `ResidentCreditLedger`** preserves credit creation + future consumption.
- **Lean `APRRecalculationBatch`** header; detail lives in superseded/replacement rows.
- **CashFlow RECONCILED (Hal & Rick correction + split-bank):** active per `(SubmissionKey, BankAccountID)` **must reconcile to SUM of allocations routed to that bank**: `2-bank receipt $500+$100, $500 void → SA bank $0 / AD bank $100`; same-bank $600 split void $500 → active $100 in that bank. Original retained as superseded. Replay never re-posts extra CashFlow.

- **Fixes F1-F4** are included; **Business Rule (accounting principle): Separate physical `CashFlow_BankID_XXX` per BankID, No Views** — 7-bank client (separate CF & P&L per bank, market increase/decrease per bank) requires isolated books; interim 6-type tables will be migrated to per-BankID. `resolveCashFlowTable(BankAccountID)` is agnostic for transition.

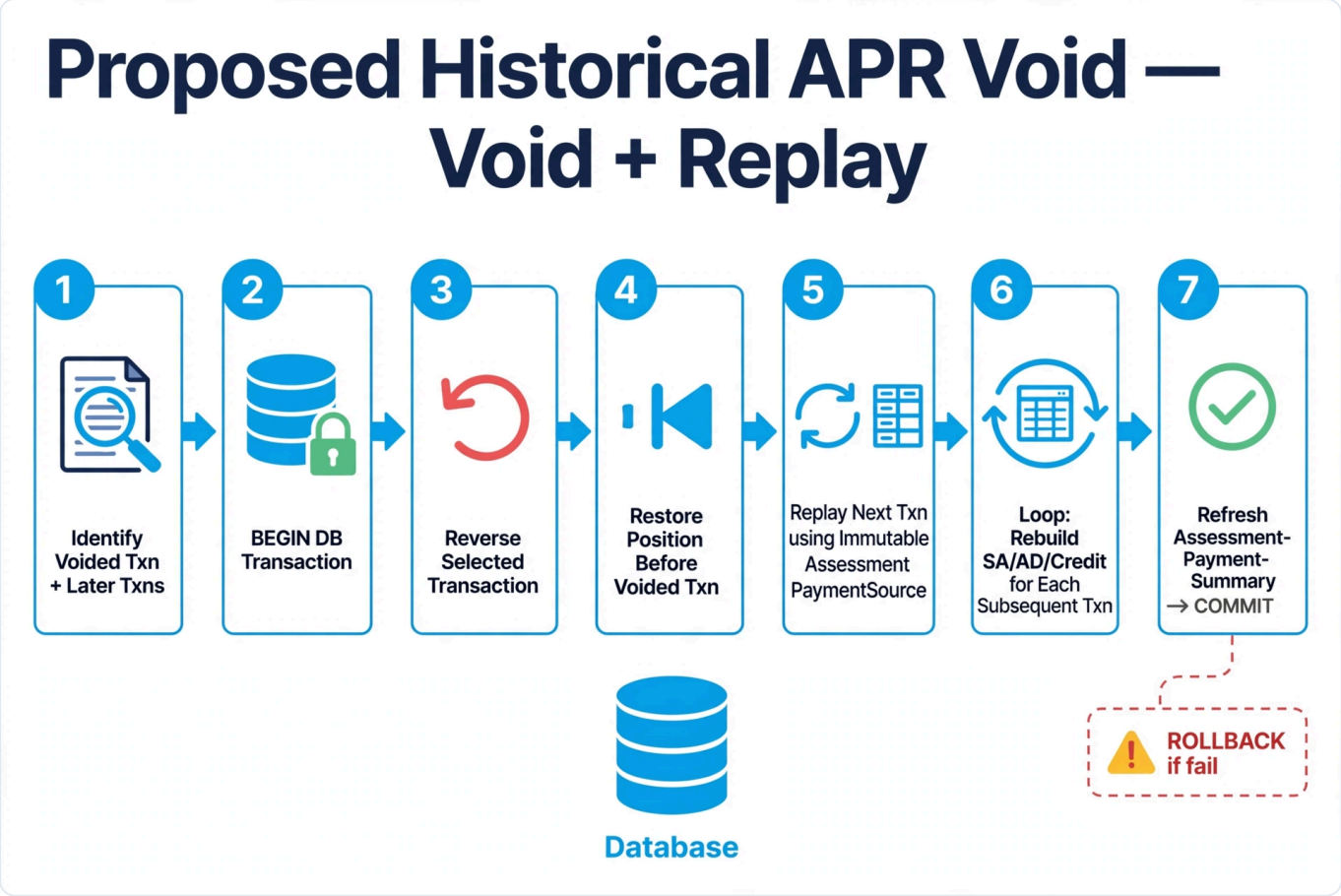

Figure 1 — `apr-void-replay_20260901-1830.png` — meta/muse-image $0.01 — 7-step atomic Void+Replay

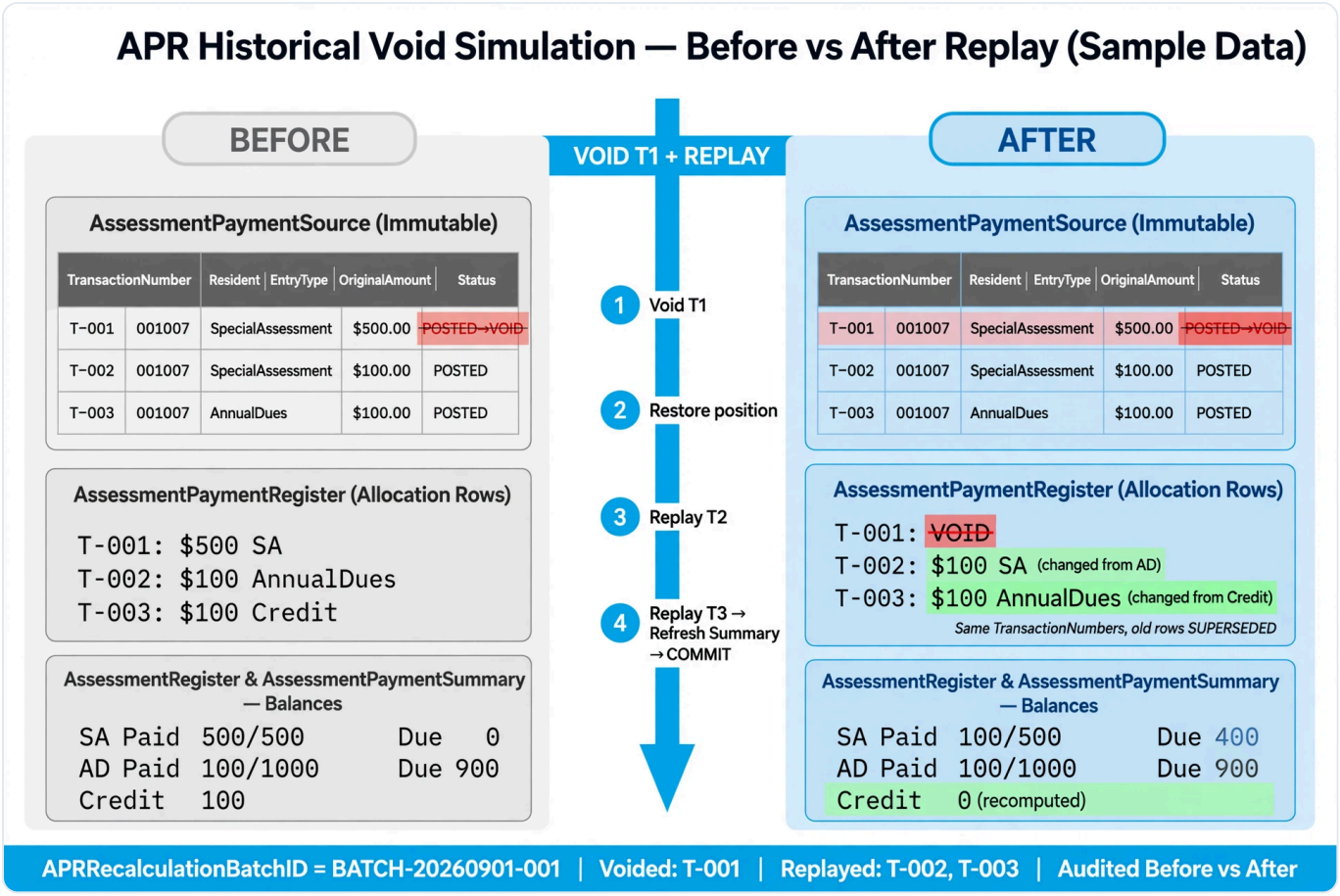

Figure 2 — `apr-void-simulacion_20260901.png` — 314KB — BEFORE T-001 $500 SA / T-002 $100 AD / T-003 $100 Credit → AFTER T-001 VOID / T-002 $100 SA / T-003 $100 AD (T-003 is AD-origin, so not $200 SA — confirmed by Rick)

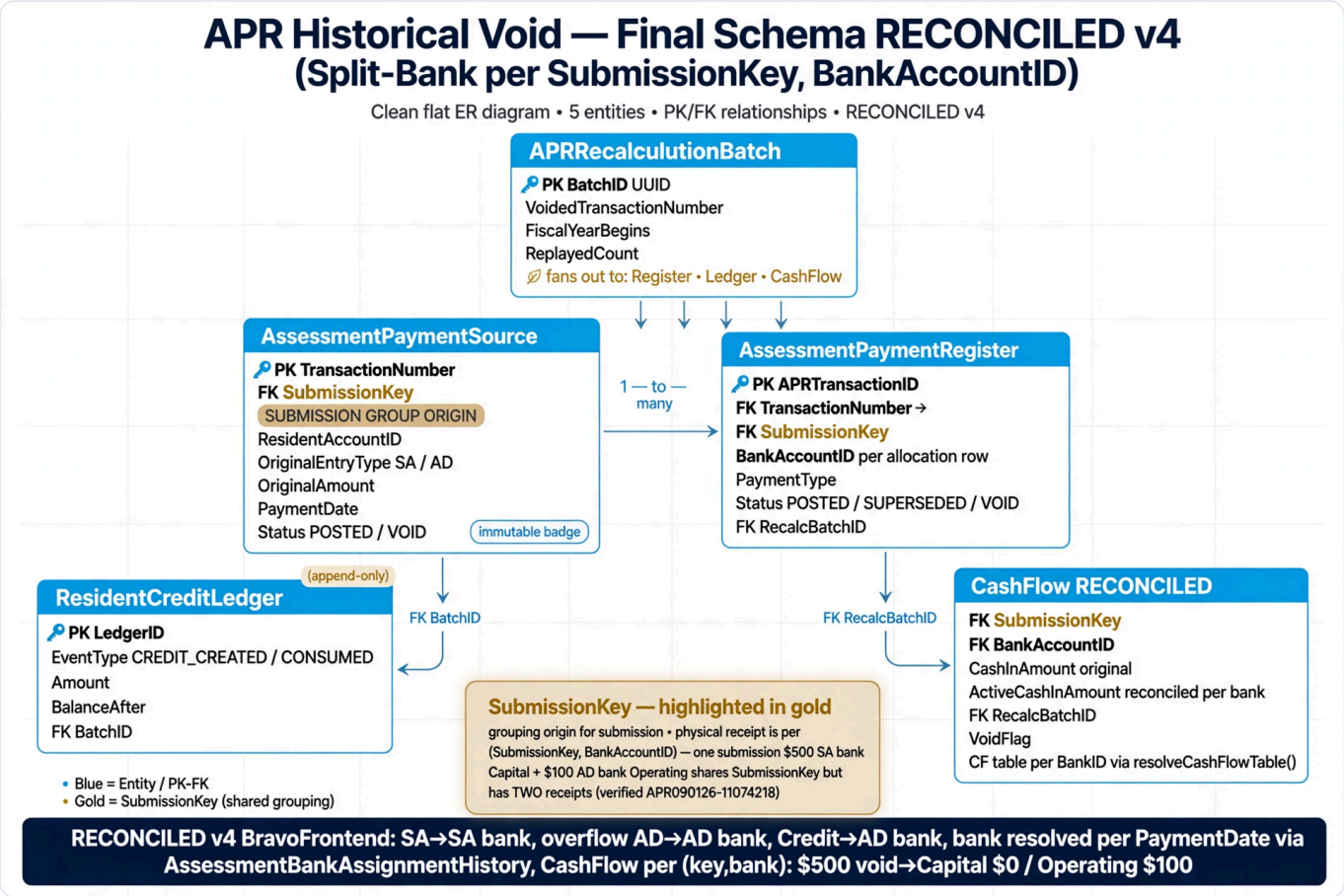

Figure 3 — `apr-void-schema-er_20260901-v4-reconciled.png` — 329KB — ER: Source → Register → CreditLedger → Batch → CashFlow via (SubmissionKey, BankAccountID) (gold). RECONCILED v4 BravoFrontend: one SubmissionKey $500→Capital + $100→Operating shares key but has TWO receipts; bank resolved per PaymentDate via AssessmentBankAssignmentHistory.

---

2. Final Schema (DDL in `002_apr_historical_void.sql`)

2.1 `AssessmentPaymentSource` — new, immutable

Column	Type	Notes
`SourceID`	INT PK AUTO	internal
`TransactionNumber`	VARCHAR(32) UNIQUE	**PK of business txn**
`SubmissionKey`	VARCHAR(36) NOT NULL	**Groups 1-2 txns sharing origin; physical receipt is per (SubmissionKey, BankID) — may be 2 receipts in 2 banks**
`ResidentAccountID`	VARCHAR(32)	
`OriginalEntryType`	ENUM SA/AD	**SA→AD→Credit vs AD→Credit**
`OriginalAmount`	DECIMAL(10,2)	as entered before allocation
`PaymentDate`, `BankAccountID`, `GLNumber`, `ElectronicPaymentID`		as entered

Column	Type	Notes
`MgtCoClientID`, `HOALicenseNumber`, `CurrentFiscalYearBegins`, `Frequency`, `PeriodNumber`		fiscal context
`OperatorID`, `Status POSTED/VOID`, `TimeStampCreated`		immutable except VOID

Indexes: `uq_source_txn`, `idx_source_submission`, `idx_source_resident_fy`, `idx_source_resident_date`.

2.2 `APRRecalculationBatch` — new, lean header

Column	Type
`BatchID`	VARCHAR(36) PK UUID
`ResidentAccountID`, `VoidedTransactionNumber`, `FiscalYearBegins`	
`OperatorID`, `Reason='historical APR Void'`, `Status COMPLETED/FAILED/IN_PROGRESS`	
`ReplayedTransactionNumbers` JSON	ordered array of later txns replayed
`ReplayedCount`, `TimeStampCreated/Completed`	

Indexes: `idx_batch_resident_fy`, `idx_batch_voided`.

2.3 `ResidentCreditLedger` — new, separate table (approved: tabla separada) — DECIDED Punto 2

Append-only, **Status + EventDate** added per Rick §2: only `POSTED` counts toward balance; replay supersedes old `SUPERSEDED`.

Column	Type	Notes
`LedgerID`	INT PK AUTO	
`ResidentAccountID`	VARCHAR(32)	
`BatchID`	VARCHAR(36) NULL FK	NULL for original; set for replay-generated
`EventDate`	DATE NULL	**DECIDED Punto 2: immutable economic date = PaymentDate of source, not replay timestamp**
`Status`	ENUM POSTED/SUPERSEDED DEFAULT POSTED	**DECIDED Punto 2: active vs superseded; balance = SUM WHERE Status='POSTED'**
`SupersededAt`, `RecalcBatchID_ledger`	DATETIME/VARCHAR(36) NULL	when superseded by replay
`EventType`	ENUM CREDIT_CREATED/CREDIT_CONSUMED	
`Amount`	DECIMAL(10,2)	positive; signed via EventType

Column	Type	Notes
`BalanceAfter`	DECIMAL(10,2)	running balance after this event
`AppliedTo`	VARCHAR(32) NULL	for CONSUMED: AnnualDues/SpecialAssessment/Fines
`ReferenceTransactionNumber`	VARCHAR(32) NULL	Source txn that created/consumed
`SubmissionKey`, `MgtCoClientID`, `HOALicenseNumber`, `CurrentFiscalYearBegins`, `OperatorID`, `TimeStampCreated`		

FK `BatchID → APRRecalculationBatch`. Indexes on resident+fy, `EventDate`, batch, `Status`. **Balance query:** `SELECT SUM(CASE WHEN EventType='CREDIT_CREATED' THEN Amount ELSE -Amount END) FROM ResidentCreditLedger WHERE ResidentAccountID=? AND Status='POSTED'`.

2.4 `AssessmentPaymentRegister` — alters

```
ADD SubmissionKey VARCHAR(36) NULL AFTER TransactionNumber
ADD RecalcBatchID VARCHAR(36) NULL AFTER Status
ADD ReplacesAPRTransactionID INT NULL AFTER RecalcBatchID
ADD SupersededAt DATETIME NULL
ADD INDEX idx_apr_submission (SubmissionKey)
ADD INDEX idx_apr_recalc_batch (RecalcBatchID)
```

Future active view: `WHERE Status='POSTED' AND DeletedFlag='N'` (migrate queries from `DeletedFlag!='Y'`).

2.5 `CashFlow_BankID_XXX` — Separate physical table per BankID, No Views (ACCOUNTING PRINCIPLE)

Business Rule: each actual `BankID` (e.g., 101 Operating, 201 Capital, 401 Money Market with market increase/decrease, plus 7-bank and 5-bank HOAs) has its own physical `CashFlow_BankID_XXX` table with `CashFlowRowControl` per `BankID` and separate CF & P&L per bank. No views. Interim 6-type tables are TEMP to be migrated.

For each **final** `CashFlow_BankID_XXX` (one per actual BankID) and **interim** `_Operating/_Capital/...` during transition:

```
ADD SubmissionKey VARCHAR(36) NULL AFTER SourceTransactionNumber -- groups origin submission
(not receipt)
ADD SubmissionTxnCount TINYINT NULL DEFAULT 1
ADD ActiveCashInAmount DECIMAL(10,2) NULL AFTER CashInAmount -- active per (SubmissionKey,Ban
kAccountID) = SUM routed to that bank; NULL means = CashInAmount
ADD RecalcBatchID VARCHAR(36) NULL AFTER SubmissionKey
ADD SupersededAt DATETIME NULL AFTER RecalcBatchID
ADD INDEX idx_cf_submission (SubmissionKey)
ADD INDEX idx_cf_submission_bank (SubmissionKey, BankAccountID)
```

Interim resolver `cfTableMap[bankType]` becomes `resolveCashFlowTable(BankAccountID) → CashFlow_BankID_<id>` for final architecture; procedure calls it so historical replay works with both. Supersede pattern like APR: original $600 in bank → `VoidFlag='Y', SupersededAt, RecalcBatchID`, replacement with `CashInAmount=activePerBank` (e.g., AD bank $100) when partially voided; `activePerBank=0` → no active row. **Split-bank note:** one `SubmissionKey` with `$500→Capital(2)` + `$100→Operating(1)` has **two receipts** — void of `$500` voids only Capital receipt, Operating $100 stays.

---

3. Backfill (proposal, not executed) — RECONCILED §8 DETERMINISTIC

1. **Source** from existing APR (17 rows dev) — **deterministic, exact TransactionNumber only** (Rick §8):

```
-- per TransactionNumber: OriginalAmount = SUM(TotalAmount)  -- SA txn with overflow su
m includes SA+AD rows sharing TransactionNumber
-- OriginalEntryType = 'SpecialAssessment' if EXISTS SA row else 'AnnualDues'
-- SubmissionKey = TransactionNumber (default 1-1); shared UUID only if exact SourceTra
nsactionNumber grouping proves 2 txns belong to same cash receipt; no heuristic residen
t+date+operator
```

2. **Register.SubmissionKey** = `Source.SubmissionKey` via exact `TransactionNumber` join; no heuristic.
3. **CashFlow.SubmissionKey** = `Source.SubmissionKey` via exact `SourceTransactionNumber = TransactionNumber`; **no** `resident+date+amount` heuristic; flag remainder for manual review.
4. **CreditLedger** initial: for each APR row with `CreditAmount>0` and `POSTED`, insert `CREDIT_CREATED` with `EventDate = PaymentDate`, `Status='POSTED'`, running `BalanceAfter`

---

4. Implementation Procedure — Historical Void + Full-Year Replay (atomic)

Preconditions (F1-F4 fixed): `allocateAprPayment(conn, source)` extracted from `enter-payment:4236-4308` and reused by both entry and replay (no duplicated allocation logic).

Endpoint: `POST /api/apr/void` extended: if `hasLaterActiveTxns(resident, fiscalYear, PaymentDate, TransactionNumber)` then historical path else latest-path (existing, after F1-F3 fixes).

Historical path — single DB transaction `db.withTransaction` — RECONCILED v6 (full-year, per Rick §§1-11):

```
-- 0) Input: transactionNumber, operatorId, MgtCoClientID, HOALicenseNumber
-- 1) Lock in order to avoid deadlock — Tenant-isolated (§7):
SELECT * FROM AssessmentRegister WHERE MgtCoClientID=? AND HOALicenseNumber=? AND ResidentAcc
ountID=? AND CurrentFiscalYearBegins=? FOR UPDATE; -- annual + special (separate)
SELECT * FROM AssessmentPaymentRegister WHERE MgtCoClientID=? AND HOALicenseNumber=? AND Resi
dentAccountID=? AND CurrentFiscalYearBegins=? FOR UPDATE;
SELECT * FROM ResidentMaster WHERE ResidentAccountID=? FOR UPDATE;
SELECT GET_LOCK(CONCAT('apr-replay:', MgtCoClientID, ':', HOALicenseNumber, ':', ResidentAcco
untID), 10);
-- Also lock DuesProgramming rows for the FY to serialize bank-program changes

-- 2) Resolve voided Source + Batch header + effective banks for PaymentDate — Tenant-isolate
d:
SELECT * FROM AssessmentPaymentSource WHERE MgtCoClientID=? AND HOALicenseNumber=? AND Transa
```

```
ctionNumber=? FOR UPDATE;
-- validate Status='POSTED', belongs to (MgtCo,HOA,Resident,FY)
-- Resolve effective banks for payDate via DuesProgramming + AssessmentBankAssignmentHistory
(payDate >= BankChangeEffectiveDate ? Pending : OldBankAccountID)
INSERT INTO APRRecalculationBatch (BatchID, MgtCoClientID, HOALicenseNumber, ResidentAccountI
D, VoidedTransactionNumber, FiscalYearBegins, OperatorID, Status)
VALUES (UUID(), ?, ?, ?, ?, ?, ?, 'IN_PROGRESS');

-- 3) Check CashFlow grouping BEFORE void — per (SubmissionKey, BankAccountID) + composite li
ne identity (§3):
-- One SubmissionKey may have 2 receipts in different banks (verified APR090126-11074218 $500
Capital + $100 Operating)
SELECT SubmissionKey, BankAccountID, GLNumber, PaymentType, CashFlowTable=resolveCashFlowTabl
e(BankAccountID)
FROM AssessmentPaymentRegister WHERE MgtCoClientID=? AND HOALicenseNumber=? AND TransactionNu
mber=? AND Status='POSTED';
-- For old-bank 45-day rule (§5): detect if any affected BankAccountID is former bank (OldBan
kAccountID with EffectiveDate <= PaymentDate) → 409 PENDING_TECH

-- 4) Mark Source VOID (immutable except this) — Tenant-isolated:
UPDATE AssessmentPaymentSource SET Status='VOID' WHERE MgtCoClientID=? AND HOALicenseNumber=?
AND TransactionNumber=?;

-- 5) RECONCILED §1 — Full-Year: Supersede COMPLETE active APR allocation state for (MgtCo,HO
A,Resident,FY) — not only target+later:
UPDATE AssessmentPaymentRegister
  SET Status='SUPERSEDED', DeletedFlag='Y', RecalcBatchID=?, SupersededAt=NOW()
  WHERE MgtCoClientID=? AND HOALicenseNumber=? AND ResidentAccountID=? AND CurrentFiscalYearB
egins=?
    AND Status='POSTED';
-- Establishes beginning-of-FY baseline: Required AD/SA obligations and Period schedules rema
in intact and separate; YTD/Credit reset to 0 before first replay (Rick §9)

-- Also supersede related CreditLedger events — RECONCILED §2:
UPDATE ResidentCreditLedger
  SET Status='SUPERSEDED', SupersededAt=NOW(), RecalcBatchID_ledger=?
  WHERE MgtCoClientID=? AND HOALicenseNumber=? AND ResidentAccountID=? AND CurrentFiscalYearB
egins=?
    AND Status='POSTED';
-- Only Status='POSTED' contributes to balance; EventDate (=PaymentDate) remains immutable fo
r replay

-- 6) Full-year rebuild from start — Tenant + PaymentDate-effective banks — RECONCILED §§7,9:
-- Fetch all Source rows for the FY that are still POSTED, ordered chronologically — Tenant-i
solated:
SELECT * FROM AssessmentPaymentSource
  WHERE MgtCoClientID=? AND HOALicenseNumber=? AND ResidentAccountID=? AND CurrentFiscalYearB
egins=? AND Status='POSTED'
  ORDER BY PaymentDate, TransactionNumber;
-- Before first replay, establish beginning-of-FY baseline per Rick §9: Required AD/SA obliga
tions and Period schedules remain intact and separate; YTD/Credit reset to 0
-- For each source IN ORDER:
--   effectiveAnnualBank = resolveAssessmentBank('annualDues', source.PaymentDate) -- via Due
sProgramming + AssessmentBankAssignmentHistory (payDate >= BankChangeEffectiveDate ? Pending
: OldBankAccountID)
--   effectiveSpecialBank = resolveAssessmentBank('specialAssessment', source.PaymentDate) --
never today's bank
--   CALL allocateAprPayment(conn, source, effectiveBanks) -- SA→SA bank, overflow→AD bank, C
redit→AD bank (AD and SA obligations never combined)
--   This inserts new APR rows (1-2 per source) with MgtCo/HOA/Resident/FY, Status='POSTED',
RecalcBatchID=?, SubmissionKey=source.SubmissionKey,
```

```
--        BankAccountID = destination bank, GLNumber = destination revenue GL, PaymentType pe
r allocation
--        Composite identity per Rick §3: (TransactionNumber, BankAccountID, GLNumber, Paymen
tType) — no LIMIT 1
--   And inserts CreditLedger rows for any overage: EventType='CREDIT_CREATED', EventDate=sou
rce.PaymentDate, Status='POSTED', BatchID=?
--   (Future: also replays CREDIT_CONSUMED chronologically with EventDate = original consumpt
ion date)

-- 7) Recompute aggregates from active rows — Tenant-isolated (no GREATEST subtract):
-- AssessmentRegister: SUM active APR rows vs required — per (MgtCo,HOA,Resident,FY,DuesType)
separate AD/SA
SELECT TotalYearlyRequiredAnnualDues, RequiredSpecialAssessment FROM AssessmentRegister WHERE
MgtCoClientID=? AND HOALicenseNumber=? AND ResidentAccountID=? AND CurrentFiscalYearBegins=?;
UPDATE AssessmentRegister SET TotalAnnualDuesPaymentsYTD=?, SpecialAssessmentPaid... , Credit
AfterPayments..., TotalCurrentAR = ... WHERE MgtCoClientID=? AND HOALicenseNumber=? AND Resid
entAccountID=? AND CurrentFiscalYearBegins=? AND DuesType=?;
-- ResidentMaster: recompute credit balance from Ledger — only POSTED counts per Rick §2:
UPDATE ResidentMaster SET ResidentCreditBalance = (SELECT COALESCE(SUM(CASE WHEN EventType='C
REDIT_CREATED' THEN Amount ELSE -Amount END),0) FROM ResidentCreditLedger WHERE ResidentAccou
ntID=? AND Status='POSTED');
-- AssessmentPaymentSummary: refresh only now — per (MgtCo,HOA,Resident)

-- 8) CashFlow — RECONCILED split-bank + composite line identity (§3) + per-bank (Hal & Rick
+ BravoFrontend §§4-5) — Tenant-isolated:
-- DO NOT re-post CashFlow during replay beyond per-bank/per-line replacement; supersede for
audit per composite key.
-- Per (SubmissionKey, BankAccountID, GLNumber, PaymentType) line: supersede original line WH
ERE SourceTransactionNumber=? AND BankAccountID=? AND GLNumber=? AND PaymentType=? AND Status
='POSTED'
-- Per (SubmissionKey, BankAccountID) bank total: activePerBank = COALESCE(SUM(amount routed
to that BankAccountID FROM active APR rows WHERE SubmissionKey=? AND BankAccountID=? AND MgtC
o/HOA/Resident/FY), 0)
--   originalPerBank = (SELECT SUM(CashInAmount) FROM <cfTable_resolved> WHERE SubmissionKey
=? AND BankAccountID=? AND MgtCo/HOA/Resident AND DeletedFlag='N')
--   cfTable = resolveCashFlowTable(BankAccountID) -- per BankID physical: CashFlow_BankID_<B
ankAccountID>, No Views
--   IF activePerBank = 0 THEN supersede all lines for that (key,bank) with VoidFlag='Y', Rec
alcBatchID=?
--   ELSIF activePerBank != originalPerBank THEN supersede original lines per composite key +
INSERT replacement line(s) per (TransactionNumber,BankAccountID,GLNumber,PaymentType) with Ca
shInAmount=portion of activePerBank
--   ELSE keep original rows.
-- For 2-bank receipt $500 Capital(2) + $100 Operating(1) sharing one SubmissionKey: void $50
0 SA → Capital $0 / Operating $100 stays; void $100 AD → Capital $500 stays / Operating $0. F
or same-bank $500+$100 split: composite distinguishes SA vs AD overflow lines even though sam
e bank.

-- 9) Finalize Batch:
UPDATE APRRecalculationBatch SET Status='COMPLETED', ReplayedTransactionNumbers=JSON_ARRAY
(...), ReplayedCount=?, TimeStampCompleted=NOW() WHERE BatchID=?;

-- 10) COMMIT (or ROLLBACK on any error) + RELEASE GET_LOCK
```

Latest-transaction path (unchanged except F1-F3 fixes): same locking, `UPDATE Source Status='VOID'`, supersede only that txn's APR rows, void CashFlow per Option A, recompute aggregates from active rows (same recompute, not subtract), no loop.

---

5. CashFlow Truth Table — RECONCILED 2026-09-01 BravoFrontend split-bank + Hal & Rick correction

Active per `(SubmissionKey, BankAccountID)` must reconcile to `SUM` of allocations **routed to that bank** (Rick §§4-5). One `SubmissionKey` with `$500→SA bank Capital(2)` + `$100→AD bank Operating(1)` (`APR090126-11074218`, verified) has **two physical receipts** — not one.

Submission + banks	Void which?	Active per (key,bank)	CashFlow rows (audit)	Replay
1 txn $600 in 1 bank	that $600	**$0**	original → VOID	No new receipt
$500 SA bank (Capital 2) + $100 AD bank (Operating 1) — 1 SubmissionKey, **2 banks**	**$500 SA void**	**Capital $0**, **Operating $100**	Capital $500→VOID, Operating $100 stays active	No new receipt
same 2-bank receipt	**$100 AD void**	**Capital $500**, **Operating $0**	Capital $500 stays, Operating $100→VOID	No new receipt
same 2-bank receipt	**both void**	**$0 + $0**	both → VOID	No new receipt
$500+$100 in **same** bank (single-bank split)	$500 void	**$100** in that bank	original $600→superseded, replacement $100 active	No new receipt
same bank	$100 void	**$500**	original $600→superseded, replacement $500 active	No new receipt

Enforced per `(SubmissionKey, BankAccountID)`:

```
activePerBank = COALESCE(SUM(routed amount to that BankAccountID FROM active APR rows WHERE S
ubmissionKey=? AND BankAccountID=?), 0)
-- cfTable = resolveCashFlowTable(BankAccountID) -- interim Map[bankType] or final CashFlow_B
ankID_XXX
-- supersede original + INSERT replacement with CashInAmount=activePerBank when 0 < activePer
Bank < original
```

DuesProgramming bank is resolved by `PaymentDate` via `AssessmentBankAssignmentHistory` (`payDate >= BankChangeEffectiveDate ? Pending : OldBankAccountID` logic as in `server.js:getAssessmentBank`), so back-dated replay never reroutes to current bank.

---

6. Concurrency & Integrity

- Ordered locks + `GET_LOCK('apr-replay:resident',10)` prevents concurrent void/replay and concurrent `enter-payment` for same resident (enter-payment locks same `AssessmentRegister` rows first).
- `FiscalYearBegins` from Source, not today — full-year rebuild uses correct FY even if void happens in later FY.
- Idempotency: `BatchID` UUID dedupes retries; second request with same `BatchID` returns existing batch.
- Isolation: `REPEATABLE READ` default; all reads inside transaction see consistent snapshot.

---

7. Audit — RECONCILED §2

`APRRecalculationBatch.BatchID` links superseded rows (`RecalcBatchID`, `SupersededAt`) and replacement rows (`RecalcBatchID`, `ReplacesAPRTransactionID`). Query — tenant-isolated:

```
-- Before vs After for a batch:
SELECT * FROM AssessmentPaymentRegister WHERE MgtCoClientID=? AND HOALicenseNumber=? AND Reca
lcBatchID=? AND Status='SUPERSEDED'; -- before
SELECT * FROM AssessmentPaymentRegister WHERE MgtCoClientID=? AND HOALicenseNumber=? AND Reca
lcBatchID=? AND Status='POSTED';     -- after (same TransactionNumbers)
-- Credit before/after — only POSTED counts, EventDate immutable:
SELECT * FROM ResidentCreditLedger WHERE MgtCoClientID=? AND HOALicenseNumber=? AND BatchID=?
AND Status='POSTED'; -- active
SELECT * FROM ResidentCreditLedger WHERE MgtCoClientID=? AND HOALicenseNumber=? AND RecalcBat
chID_ledger=? AND Status='SUPERSEDED'; -- superseded
```

No duplication of allocation detail in batch header; header stores ordered `ReplayedTransactionNumbers` JSON for quick trace.

---

8. Testing Plan — RECONCILED per Rick §4 (OriginalEntryType explicit, per-bank receipts)

- **Unit:** `allocateAprPayment` determinism per `OriginalEntryType` (`SA→AD→Credit` vs `AD→Credit`).
- **Integration — canonical with OriginalEntryType explicit (Rick §4 + Figure 2):**
  - **Case A (Figure 2, same SA-origin chain):** T-001 `SA $500` (SA) → SA, T-002 `SA $100` (SA) → SA (overflow after SA full), T-003 `SA $100` (SA) → Credit; void T-001 → full-year replay → T-002 `SA $100` now → **SA** (SA still due, AD not touched) — validates SA-origin follows SA.
  - **Case B (AD-origin stays AD):** T-001 `SA $500` → SA, T-002 `AD $100` (AD) → AD, T-003 `AD $100` (AD) → Credit; void T-001 → T-002 `AD $100` stays **AD** (AD-origin never routes to SA, even though SA again due) — validates AD-origin immutability. Both cases test same `TransactionNumber` but different `OriginalEntryType`.
- **Split-bank receipts — two cases (Rick §4):**
  - **Different banks:** UF `SA $500 → SA bank Capital(2) + AD $100 → AD bank Operating(1)` (verified 001006) → **2 receipts** `$500 Capital + $100 Operating`; void `$500` → active `Capital $0 / Operating $100` (separate tables).
  - **Same bank:** `SA $500 + AD $100` both to `Operating 1` via same program → **1 receipt $600** split as 2 allocation lines but same `BankAccountID`; `SUM per (SubmissionKey, BankID)` still $600 but composite `(GL,PaymentType)` distinguishes lines.
- **Multi-row SA overflow:** `APR083126-12381556: $264.23 SA + $36.54 AD` → void → full rebuild produces same allocation (no under-reverse, F1).
- **Credit ledger:** create overage → verify `Ledger CREDIT_CREATED` with `EventDate=PaymentDate` and `Status='POSTED'`; consume via `CREDIT_CONSUMED` → replay preserves both with `Status` filter and `EventDate` immutable.
- **Concurrency + tenant:** two posts for same `(MgtCo,HOA,Resident)` during replay → blocked by `GET_LOCK(MGT:HOA:Resident)`; second void → rejected.

- **Invariant checker:** `AssessmentRegister` totals == `SUM(active APR WHERE Status='POSTED')` per `(MgtCo,HOA,Resident,FY,DuesType)` separate AD/SA; `ResidentMaster.CreditBalance == SUM(Ledger WHERE Status='POSTED')`.

---

9. Next Step — Awaiting Mutual Approval

No DDL or code will be executed or committed until you approve this final procedure in writing. Upon approval, execution order will be: (1) run `002_apr_historical_void.sql`, (2) backfill, (3) land F1-F4 fixes, (4) implement `allocateAprPayment` extraction + historical `void` endpoint with full-year loop, (5) land `refreshAssessmentPaymentSummary` recompute.

Decisiones Jose 2026-09-01 — Puntos 3,5,6 (a solicitud tuya):

- **Punto 3 — CF row identity: Clave compuesta `(TransactionNumber, BankAccountID, GLNumber, PaymentType)`**, sin nuevo `AllocationLineID` (usa misma clave que `server.js:4790 groups`; cubre SA $500 + AD $100 mismo banco/SubmissionKey pero distinto GL/PaymentType; `LIMIT 1` eliminado).
- **Punto 5 — Old-bank 45-day:** Void normal **bloqueado con 409 Requires Technical Assistance** si alguna fila afectada está en banco viejo (`BankAccountID = OldBankAccountID` con `EffectiveDate <= PaymentDate` y es distinto al banco programado hoy) — dinero permanece en banco viejo, moverlo requiere transferencia separada post-45d, no reescritura.
- **Punto 6 — Identificador CashFlow: `BankAccountID` (PK interno, ej. 101) es inmutable para `CashFlow_BankID_XXX`**, no `BankID` string editable; provisión automática `CREATE TABLE CashFlow_BankID_<BankAccountID> LIKE Template` + `CashFlowRowControl` por `BankID` al crear `BankAccount`.

Prepared by Jose — 2026-09-01 RECONCILED v4 + Decisions 3,5,6 — For Discussion Only.

Attachments: `002_apr_historical_void.sql` v4-nv (18K, `d1937ff`, separate per BankID No Views), `decisiones-puntos-3-5-6_20260901.md` (decisiones), Figures 1-3 (Figures 1-2 unchanged, Figure 3 v4-reconciled 329K), `why-views` PDF superseded (alternative not adopted).

Reconciliation notes vs v3: `SubmissionKey` ≠ 1 receipt; CashFlow per `(SubmissionKey, BankID)`; bank per `PaymentDate` via History; `CashFlow_BankID_XXX` per BankID No Views; CreditLedger `EventDate` + `Status` per Punto 2; composite key per Punto 3.
